PROTOFIRE

May 2026

Smart contract audit

st0x.deploy

Content

Project description

This repository implements st0x-specific deployment and extension contracts for the rain.vats OffchainAssetReceiptVault system. It provides a st0x receipt vault, ERC-1155 receipt implementation, corporate-action facet, ERC-4626 wrapper vault, authorizers, and deployment helpers for tokenised real-world assets.

The core st0x extension is stock-split-aware rebasing for both ERC-20 vault shares and ERC-1155 receipt balances. Corporate actions are scheduled through a delegatecalled facet, represented in a chronological linked list, and applied lazily on balance reads and first account interaction after the action becomes effective.

The repository also includes beacon-proxy deployment infrastructure for standardised upgradeable deployments, including receipt vault and wrapped-token vault deployment paths. The ERC-4626 wrapper captures rebases in share price for DeFi compatibility while preserving the underlying receipt-vault accounting model.

Executive summary

Type	Vault
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/S01-Issuer/st0x.deploy

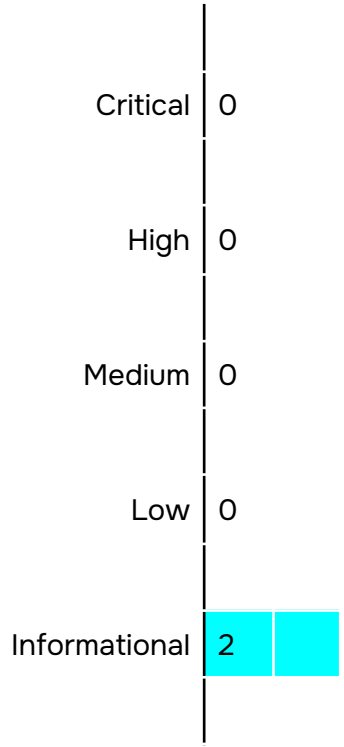
Reviews

Date	Repository	Commit
22/01/26	st0x.deploy	5658a076a8ff1f83909c829c3b026b0097ada239
26/01/26	st0x.deploy	58877e0ffe20fbb5ce02d3fb9dcff923067447c0
11/05/26	st0x.deploy	0f04fd9c5899474268626f302f41a0e2b898af51
12/05/26	st0x.deploy	ce0b3dfc46490614f1401b45343370e24a07c59d
21/05/26	st0x.deploy	da1dd6cd6d30c2d1c5b030d6a9d9dc27df520082 **tag v0.1.1**

Scope

Contracts and their dependencies
src/generated/StoxCorporateActionsFacet.pointers.sol
src/generated/StoxOffchainAssetReceiptVaultAuthorizerV1.pointers.sol
src/generated/StoxOffchainAssetReceiptVaultBeaconSetDeployer.pointers.sol
src/generated/StoxWrappedTokenVault.pointers.sol
src/generated/StoxReceipt.pointers.sol
src/generated/StoxWrappedTokenVaultBeacon.pointers.sol
src/generated/StoxWrappedTokenVaultBeaconSetDeployer.pointers.sol
src/generated/StoxUnifiedDeployer.pointers.sol
src/generated/StoxReceiptVault.pointers.sol
src/generated/StoxOffchainAssetReceiptVaultPaymentMintAuthorizerV1.pointers.sol
src/interface/IStoxUnifiedDeployerV1.sol
src/interface/IStoxWrappedTokenVaultBeaconSetDeployerV1.sol
src/interface/ICorporateActionsV1.sol
src/concrete/StoxReceipt.sol
src/concrete/deploy/StoxUnifiedDeployer.sol
src/concrete/deploy/StoxWrappedTokenVaultBeaconSetDeployer.sol
src/concrete/deploy/StoxOffchainAssetReceiptVaultBeaconSetDeployer.sol
src/concrete/StoxWrappedTokenVault.sol
src/concrete/authorize/StoxOffchainAssetReceiptVaultAuthorizerV1.sol
src/concrete/authorize/StoxOffchainAssetReceiptVaultPaymentMintAuthorizerV1.sol
src/concrete/StoxWrappedTokenVaultBeacon.sol
src/concrete/StoxCorporateActionsFacet.sol
src/concrete/StoxReceiptVault.sol
src/lib/LibCorporateActionReceipt.sol
src/lib/LibTotalSupply.sol
src/lib/LibProdDeployV2BaseOverrides.sol
src/lib/LibProdDeployV1.sol
src/lib/LibProdDeployV3.sol
src/lib/LibProdDeployV2.sol
src/lib/LibERC1155Storage.sol
src/lib/LibERC20Storage.sol
src/lib/LibReceiptRebase.sol
src/lib/LibCorporateAction.sol
src/lib/LibProdTokensBase.sol
src/lib/LibRebaseMath.sol
src/lib/LibRebase.sol
src/lib/LibCorporateActionNode.sol
src/lib/LibStockSplit.sol
src/error/ErrRebase.sol
src/error/ErrCorporateAction.sol
src/error/ErrStockSplit.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No medium severity issue found.

* Low

No low severity issue found.

Informational

I01 - Unused declared named return variable

The `effectiveTotalSupply()` function in `LibTotalSupply` declares a named return variable `supply`, but the variable is never assigned or used within the function body. Instead, a local variable `running` serves as the accumulator and is returned via an explicit return `running` statement.

Impact: Reduced readability and a latent maintenance hazard for future modifications of the function.

Path: src/lib/LibTotalSupply.sol

Recommendation: Consider removing the name from the return declaration.

Found in: 0f04fd9

Status: Fixed at aba8db0

I02 - Cancelled nodes retain parameters and remain readable by raw cursor

The `cancel()` function zeros only `prev`, `next`, and `effectiveTime`. The `actionType` and `parameters` survive. Correct list consumers walk via head-tail and never see cancelled nodes, but `getActionParameters(actionId)` on the facet returns the old payload for a cancelled id.

Path: src/lib/LibCorporateAction.sol

Recommendation: Consider documenting intended behaviour.

Found in: 0f04fd9

Status: Fixed at 87ce4d1

General Risks and Assumptions

- The `receiptInformation()` function in the `Receipt` contract can be invoked at any time by any user. However, there is no specific event type associated with this function. This can lead to potential confusion when reading and interpreting events, as it may not be clear which function invocation triggered a particular event.
- Confiscating shares from the `StoxWrappedTokenVault` contract is possible via `OffchainAssetReceiptVault.confiscateShares(confiscatee, targetAmount, data)`, callable by any address that the authorizer grants the `CONFISCATE_SHARES` permission to. If the vault contract itself is ever the `confiscatee` (i.e., it holds shares as ERC4626 assets), those shares can be transferred to the confiscator and break the logic of the `StoxWrappedTokenVault`. This behavior is part of the documented confiscation design and is intended to be controlled by the `authorizer`, but we still note it here as an operational assumption.
- `StoxReceipt._update()` migrates each `(holder, id)` pair before `super._update()` fires ERC-1155 hooks. Migration walks the vault's corporate action list via `STATICCALL` to `nextOfType` and `getActionParameters()`, trusting each individual response without snapshotting the full walk into memory first. The receipt has no defence against a malicious vault implementation installed via beacon upgrade — such an implementation could serve inconsistent answers across the loop iterations, causing balance inflation, zeroing, or arbitrary drift on first-touch. The protocol therefore assumes the beacon owner (`BEACON_INITIAL_OWNER = rainlang.eth`) installs only audited vault implementations behind every receipt's manager pointer.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

	**Likelihood** Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	**Impact** Measures the technical loss and business repercussions resulting from a successful attack
	**Severity** Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	**Advanced DeFi Scrutiny** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	**Semantic Consistency Checks** We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	**Security Analysis** The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.